



REDLINE DAYLIGHT · AUTOMATION HANDBOOK

The Complete Guide to Learning Ansible

Your complete reference to **Ansible** — the Architecture, installing on every OS, setting up SSH Keys, Modules and the Inventory, Ad-hoc Commands, YAML fundamentals, full Playbooks with result reading, and Roles with the Galaxy platform. Based on the course by the **Cloud Code** channel — transcribed, corrected, and organized into a complete written reference. All technical terms kept in English exactly as used in the industry.

By **Nabawy**

AUTHOR & COMPILER

REDLINE EDITION

Daylight · A4-Fixed

SSH

Secure Access

YAML

Playbook Language

Roles

Proper Organization

Table of Contents

	The Map — Executive Summary	3		Inventory — Your Server Map	11
	Intro — What is Ansible?	4		Ad-hoc Commands — Instant Execution	12
	Architecture — Two Components	5		YAML Basics — The Language	13
	Comparison — Manual vs Bash vs Ansible	6		Playbooks — Write & Run	14
	Use Cases — Where Ansible Shines	7		Playbooks — Removal & Results	15
	Installation — Every OS	8		Roles & Galaxy — Organization	16
	SSH Keys — Passwordless Access	9		Test Yourself — 12 Questions	17
	Modules — The Execution Units	10		Answer Key	18
				Cheat Sheet	19

► **TASK [Status Color Legend]** how to read results

 **ok** — server already in the desired state, no change  **changed** — a change was applied successfully

 **failed** — a task errored out  **RED** — headings & structure (visual identity)

[**DEBUG**] **How to use this book:** 12 connected chapters — each one opens from the previous and sets up the next. Every technical term stays in English exactly as used in the industry. Code reads left-to-right in natural code direction. Every line from the source course is here — compare freely.



The Map — The Whole Route

Before touching a single command, here's the whole story — every sentence below is a link that takes you to its chapter.

We start with an existential question: **why do we bother logging into every server manually?** The answer begins in [Chapter 1 — Intro](#): Ansible is an **Automation** tool, **Open-Source**, **Agentless**, with simple **YAML** syntax, a **Declarative** approach, and safe repeatability — **Idempotent**. Then in [Chapter 2 — Architecture](#) we see the whole picture in just two components: a **Control Node** whispering over an **SSH Connection** with a **Public/Private Key Pair** to the **Managed Nodes** (Ubuntu / CentOS / DB).

In [Chapter 3 — Comparison](#) we weigh Ansible against the Manual way and Bash scripts on 3 different-distribution servers (Ubuntu, CentOS, Alpine) to install Nginx. In [Chapter 4 — Use Cases](#) we tour the three domains: **Configuration Management**, **Infrastructure Provisioning** (where the Terraform conversation happens), and **Network Automation**.

Now the practical gear: in [Chapter 5 — Installation](#) we install on macOS, Ubuntu, Pip and WSL, and set up the VS Code extensions. In [Chapter 6 — SSH Keys](#) we generate the Key Pair with `ssh-keygen` and push the Public Key with `ssh-copy-id` — the foundation every automation after it stands on.

Language and execution: in [Chapter 7 — Modules](#) we meet apt/yum, ping, service, copy and command/shell. In [Chapter 8 — Inventory](#) we write hosts.ini with groups. In [Chapter 9 — Ad-hoc Commands](#) we test and execute instantly. In [Chapter 10 — YAML](#) we master Indentation and the data structures.

The peak: in [Chapter 11 — Playbooks](#) we write a complete Apache install-and-run Playbook, run it, and read the Recap — and in [Chapter 11.b](#) we reverse it with removal. Finally in [Chapter 12 — Roles & Galaxy](#) we organize and share. Close with the [Quiz](#) and pin your knowledge with the [Cheat Sheet](#). The route is clear. ►

KEY IDEA **The unifying idea:** you write the Desired State once — Ansible enforces it across all your Managed Nodes. And if no change is actually needed, it touches nothing.

Intro – What is Ansible?

Ansible is an **Open-Source Automation** tool used primarily for two things:

1 Configuration Management

Managing settings — installing and updating software and configuring files across your servers.

2 Infrastructure Provisioning

Building and preparing infrastructure — creating servers and cloud resources.

► **TASK [Ansible's Four Core Features]** the four pillars

1.1 Agentless

- 📦 No need to install any software or **Agents** on the target **Managed Nodes**.
- 🔑 Connection happens over **SSH** (on Linux systems) or **WinRM/SSH** (on Windows systems).
- 🐍 The only requirement on Linux servers is a **Python** environment.

1.2 Simple Syntax

All configurations and automation scenarios are written in **YAML** — a language that is very simple to read and write, easy to learn and execute. No complex brackets, no semicolons; just spaces and order.

1.3 Declarative Approach

You define the **Desired State** in the YAML file — like: "I want Apache and Git installed" — and Ansible executes the steps needed to reach that state on its own. It also supports the **Imperative Approach** when using **Ad-hoc Commands**.

1.4 Idempotency

Running the same script multiple times in a row **changes nothing unless a change is actually needed**.

[EXAMPLE] If you ask Ansible to install Apache and it's already installed at the required version → it won't reinstall it and won't touch the server. The status shows **ok**. Not installed? It installs it → **changed**.

Agentless**Simple YAML****Declarative****Idempotent**

► How is this tool built internally? Two components only — **Chapter 2: Architecture**



Architecture – Two Components

Ansible's architecture consists of **two main components** — deliberate simplicity that is the secret of its power.

[Architecture – the two components and the channel]



The channel: a **Public/Private Key Pair** — secured connection, no passwords.

2.1 Control Node

The machine or server where **Ansible** is installed, and through which the **DevOps** engineer runs commands and **Playbooks**. It can be a personal laptop or a Linux server in a lab or the cloud.

2.2 Managed Nodes (Targets)

The servers targeted for applying changes and configurations. They receive instructions over **SSH** and execute them **without needing Agents installed** — that's why the architecture is called Agentless.

[**KEY IDEA**] Notice what doesn't exist: no central database, no daemon constantly running on the targets, no agent to update. Ansible connects when needed, executes, then leaves. The footprint on Managed Nodes = zero.

► TASK [Component Map] who does what

Component	What is it?	What's installed on it?	Its role
Control Node	Laptop / Linux Server	Ansible (only)	Running commands and Playbooks
Managed Nodes	Ubuntu / CentOS / DB...	Nothing — Python only	Receiving SSH commands & executing

[**WATCH OUT**] Common beginner mistake: installing Ansible on the Managed Nodes "just in case." Don't — they don't need it at all. The Control Node is the only place installation belongs (more in the Installation chapter).

🎓 **EXAM** Interview question: "Describe Ansible's architecture?" → **Two components**: a Control Node (has Ansible, runs everything) + Managed Nodes (receive over SSH, no Agents), and the channel between them is a Key Pair. Architectural simplicity = operational ease.

▶ The architecture is clear — but how does it beat traditional methods? A practical comparison on 3 servers → [Chapter 3: Comparison](#)



Comparison – Manual vs Bash vs Ansible

Imagine you're an engineer managing **3 Linux servers** with different distributions — **Ubuntu, CentOS, Alpine** — and you want to install **Nginx** on all of them. Three paths are open to you, and the difference between them is a whole philosophy:

Aspect	👉 Manual	📄 Bash Scripts	▶ Ansible
How it works	SSH into every server and run commands by hand.	Write a Bash script that detects the OS and runs the commands.	Write one simple YAML file defining the Desired State for everyone.
Speed & effort	Slow and full of mistakes.	Centralized but requires high scripting expertise.	Fast, centralized, and Simple Syntax.
Idempotency	Not available — manual double-checking.	Not automatic — conditions written by hand.	Automatically supported ✓
Distribution support	Requires knowing every distro's commands (<code>apt</code> , <code>yum</code> , <code>apk</code>).	Requires writing <code>if/else</code> for every OS.	Modules handle all distros automatically ✓

[ASIDE • The paradox] Notice that Bash scripts are smarter than Manual — but they accumulate: every new distribution = a new `if/else` branch, every update = reviewing every script. Ansible flips the equation: you don't write **how** to execute — you write **what** you want, and it knows the How for every distro inside the Modules.

► TASK [Reading the table practically] real-world lens

By real standards: to install Nginx manually on the three, you'd need 3 SSH sessions and 3 different commands (`apt install` / `yum install` / `apk add`) plus manual verification on each. With Bash: one script, but with branching for every package manager. With Ansible: **one Task** — `package: name=nginx state=present` — works on all three, and Idempotency verifies itself automatically on every future run.

[🎓 EXAM] Memorize the pairs: Manual = slow + errors + no Idempotency · Bash = centralized but heavy scripting + manual conditions · Ansible = **centralized YAML + automatic Idempotency + Modules that handle distros**. This exact comparison is a recurring interview question.

▶ Convinced of the advantage — where is it actually used? The three major domains → **Chapter 4: Use Cases**

Use Cases – Where Ansible Shines

Three major arenas where Ansible works today — the first is the most famous, the second carries a well-known industry debate.

4.1 Configuration Management

The most famous and important use: installing and updating **Packages**, adjusting **Files**, and updating Servers from one place — whether the servers sit in a local Datacenter or on the Cloud. Hundreds of servers, one command.

4.2 Infrastructure Provisioning

Creating and destroying cloud resources (**EC2, S3, Containers**) through the **APIs** of cloud services (AWS, Azure, GCP).

[ASIDE • Industry note] Many people prefer using **Terraform** for Provisioning (building the infrastructure) and **Ansible** for Configuration Management (managing what's inside the servers). Division of labor: the contractor builds the house, the technician wires the electricity. 🏠🔧

4.3 Network Automation

Configuring and managing **Routers, Switches**, and **Firewalls** programmatically through automation protocols — instead of logging into every network device by hand through its interface.

► TASK [The Quick Decision Map] which tool when?

Your task	Right tool	Why
Install Nginx on 20 servers	Ansible	Configuration inside machines — its first domain
Create a VPC and 10 instances on AWS	Terraform	Cloud provisioning over API
Full project: build + configure	Both 🍷	Terraform builds → Ansible configures
Update firmware on network switches	Ansible	Network Automation — its third domain

Notice: the comparison isn't "best vs worst" — it's "the tool in its domain." Ansible can do Provisioning, but Terraform is stronger there.

🎓 **EXAM** Memorize the trio in order: **Configuration Management** (most famous) · **Infrastructure Provisioning** (cloud, with the Terraform debate) · **Network Automation** (routers/switches/firewalls). "What are Ansible's use cases?" — this is the model answer.

▶ Domains are clear — now the gear: installing on every OS → [Chapter 5: Installation](#)



Installation – Setup on Every OS

Golden rule before we start: Ansible is installed **only on the Control Node** — a rule with no exceptions. Pick your path by OS:

macOS — Homebrew

```
$ brew install ansible
```

Ubuntu / Debian — apt

```
$ sudo apt update && sudo apt install ansible -y
```

Any OS — Python Pip

```
$ pip3 install ansible
```

Windows — WSL

Use **WSL** (Windows Subsystem for Linux) to run Ubuntu inside Windows and install Ansible on it naturally — then apply the Ubuntu command above.

► TASK [Verify the Installation] it works

```
$ ansible --version
ansible [core 2.16.3]
  config file = /etc/ansible/ansible.cfg
  python version = 3.10.12
```

A version number appeared = installation succeeded and you're ready.

► TASK [Recommended VS Code Extensions] editor setup

YAML Extension – Red Hat

Ansible Extension – Red Hat

Syntax highlighting and validation inside the editor — catches Indentation errors before they reach a Run.

[WATCH OUT] Common mistake: installing Ansible on the Managed Nodes "just in case." Useless — they only receive commands over SSH. Installation belongs to the Control Node alone.

[DEBUG] `pip3 install` is the shared option for every OS — especially useful if your distro's package manager ships an outdated version.

EXAM Three install methods + Windows: `brew` (macOS) · `apt` (Ubuntu/Debian) · `pip3` (Any OS) · **WSL** (Windows). And "Where is Ansible installed?" — **the Control Node only**.

 The tool is ready — but connecting to servers needs securing: the SSH Keys → [Chapter 6](#)



SSH Keys – Passwordless Secure Setup

For the Control Node to connect to the Managed Nodes and execute instructions **automatically**, the connection should be set up over SSH keys — the **Public/Private Key Pair**.

[Play – three steps and the connection is done]

1 **ssh-keygen**
generate the Pair



2 **ssh-copy-id**
push the Public Key



3 **Test**
no Password ✓

6.1 Generate the Key Pair on the Control Node

```
$ ssh-keygen -t rsa -b 4096
```

This creates two files in the `~/.ssh/` folder:

🔑 **id_rsa — Private Key**

The **Private Key** — **kept strictly secret** on the Control Node, never shared with anyone, ever.

🔑 **id_rsa.pub — Public Key**

The **Public Key** — this is the one that **gets pushed** to the Managed Nodes. Its lock opens nothing.

6.2 Copy the Public Key to the Managed Node

```
$ ssh-copy-id -i ~/.ssh/id_rsa.pub ubuntu@<MANAGED_NODE_IP>
```

6.3 Test the Passwordless Connection

```
$ ssh ubuntu@<MANAGED_NODE_IP>
Welcome to Ubuntu 22.04 LTS ... # logged in – no password asked ✓
$ exit
```

[**DEBUG • AWS EC2**] When using AWS EC2 machines, you don't generate a new key — use the `.pem` Private Key file provided by AWS to format the first connection (the User is usually `ubuntu` or `ec2-user` depending on the OS).

[**WATCH OUT**] Two common mistakes: copying `id_rsa` (the Private) instead of `id_rsa.pub` (the Public) to the server = the connection won't work. And pushing the Private over insecure channels = a complete security hole. The Public travels; the Private never moves.

🔥 **EXAM** The literal sequence: `ssh-keygen -t rsa -b 4096` ← `ssh-copy-id -i ~/.ssh/id_rsa.pub ubuntu@IP` ← test with `ssh ubuntu@IP`. From this moment, every upcoming Ansible command is completely silent — no password between you and automation.

▶▶ The channels are open — now we hand Ansible its tools and its map: **Chapter 7: Modules**

Modules – The Execution Units

All of Ansible's work depends on **Modules** — ready-made software tools that perform specific functions. You don't write scripts; you call Modules and hand them the desired State.

► **TASK [The most famous core Modules]** know these five

Module	What it does
<code>apt</code> / <code>yum</code>	Managing, installing, updating, and removing Packages (depending on the distribution).
<code>ping</code>	Testing connectivity and readiness between the Control Node and the Managed Nodes.
<code>service</code>	Managing Services — start, stop, restart.
<code>copy</code>	Copying files from the Control Node to the Managed Nodes.
<code>command</code> / <code>shell</code>	Running direct system commands on the servers.

There are hundreds of Modules covering almost everything — and you can always refer to the official **Ansible Documentation** to find the right Module and its options for each task.

► **TASK [The three Modules you'll use first]** start here

ping

The first Module you'll try — proves the connection works. The success response is always: `"pong"`.

apt / yum

The heart of package management. With `state=present` it becomes Idempotent automatically.

service

Controls a Service's life: `started`, `stopped`, `restarted` — plus `enabled` at boot.

[**KEY IDEA**] The Module hides the How from you: you say `apt: name=nginx state=present` — and the Module itself knows whether to actually run `apt-get install` or that everything already exists and skips it. This is the root of Idempotency.

[**WATCH OUT**] `command` and `shell` can be a trap: they let you bypass all of Ansible's elegance and write raw non-Idempotent commands. Use them as a last resort when no dedicated Module exists for your task.

🎓 **EXAM** Memorize the core five with their functions: `apt/yum` (Packages) · `ping` (Connectivity) · `service` (Services) · `copy` (Files) · `command/shell` (Direct commands). "Which Module do you use for X?" — the answers come from this list.

🔧 Tools ready — now we need the target list: who are we executing on? → [Chapter 8: Inventory](#)

Inventory – Your Server Map

The **Inventory** file is the file that holds the **IPs** of the Managed Nodes — divided into Groups that you target in commands.

► TASK [Example hosts.ini file] inventory example

```
# hosts.ini
[web]
192.168.1.101 ansible_user=ubuntu
192.168.1.102 ansible_user=ubuntu

[db]
192.168.1.201 ansible_user=ubuntu
```

[**DEBUG · File anatomy**] `[web]` and `[db]` = group names in square brackets · the lines under them = member IPs · `ansible_user=ubuntu` = the SSH user used to connect to that server — defined once here instead of repeated in every command.

► TASK [How do we target?] targeting

`[web]` → 2 Servers `[db]` → 1 Server `all` → 3 Servers

`ansible web ...` hits the web servers only · `ansible all ...` hits everyone · target by group instead of stuffing IPs manually every time.

[**KEY IDEA**] The Inventory is the "source of truth" for your infrastructure map: added a server? One line in the file. Changed a user? One edit. Every command and Playbook coming next will refer to this file via `-i hosts.ini`.

[**WATCH OUT**] Group names are strictly case-sensitive: `[web]` is not `[webs]` and not `[Web]`. One wrong letter in targeting = "no matching hosts" — and the error message won't hint at your typo.

🎓 **EXAM** Inventory = a file holding Managed Node IPs divided into Groups · Syntax: `[group_name]` then the lines · `ansible_user` defines the SSH user · always passed with `-i`.

▶ We have the Modules and the Inventory — time for real execution, with no file at all: → [Chapter 9: Ad-hoc Commands](#)

⚡ Ad-hoc Commands – Instant Execution

Ad-hoc Commands are used to run a **quick task without writing a full Playbook** — a single line straight from the terminal. This is the Imperative side we mentioned in Chapter 1.

► TASK [The general syntax] anatomy

```
ansible <group_or_all> -i <inventory_file> -m <module_name> -a "<module_arguments>"
```

► TASK [Three practical examples] from ping to install

1 Test connectivity — ping all servers:

```
$ ansible all -i hosts.ini -m ping
192.168.1.101 | SUCCESS => { "ping": "pong" }
192.168.1.102 | SUCCESS => { "ping": "pong" }
192.168.1.201 | SUCCESS => { "ping": "pong" }
```

2 Get the Uptime of web servers — the web group:

```
$ ansible web -i hosts.ini -a "uptime"
```

[**DEBUG**] Smart note: when `-m` is omitted, the `command` Module is used **by default** — that's why we wrote `uptime` directly in `-a` without naming a module.

3 Ensure Nginx is installed on all servers:

```
$ ansible all -i hosts.ini -m apt -a "name=nginx state=present" --become
```

[**KEY IDEA**] `--become` = root privileges (like sudo) — installation always needs them. And `state=present` is the language of Idempotency: "make sure it exists" — already exists? Nothing happens at all.

[**WATCH OUT**] Ad-hoc or Playbook? Ad-hoc is for a task that's **quick and won't repeat** — a check, a quick restart, a query. The moment you feel the command will be reused or shared with the team → move it to a Playbook immediately (next chapter).

🎓 **EXAM** Memorize the structure in order: **Target** ← **Inventory (-i)** ← **Module (-m)** ← **Arguments (-a)** ← **Become**. And the classic comprehension question: "Why didn't we write `-m command` in the uptime example?" — because it's the default Module.

▶▶ Single commands do one task — full scenarios need an organized file. Its language first: → [Chapter 10: YAML](#)

YAML Basics – The Language

YAML is a declarative language that depends entirely on **Indentation** — not a programming language, just an orderly way to describe data. Mastering its rules = writing Playbooks without headaches.

[**WATCH OUT** • The most important rule] Never use **Tab** — always use **two Spaces**. Mixing Tab with Spaces is the #1 cause of failed YAML files for beginners, and the error appears far from its real line, making it hard to trace.

► **TASK [The first rule]** document start

Files always start with three dashes `---` — declaring the start of the YAML document:

```
---
```

► **TASK [The three data structures]** everything is built from these

① Key-Value

```
name: Ahmed
```

Key and value — the simplest structure.

② Dictionary

```
employee:
  name: Mohamed
  age: 30
  is_manager: true
```

Dictionary — multiple keys under a parent key.

③ List

```
skills:
  - Linux
  - Ansible
  - Docker
```

List — a dash per item.

[**KEY IDEA**] The Playbook we'll write in the next chapter is nothing new — it's exactly a **List of Dictionaries** (structure ④): each Play is an item in a List, and each Task inside it is a Dictionary. Whoever understands these three structures understands all of Ansible.

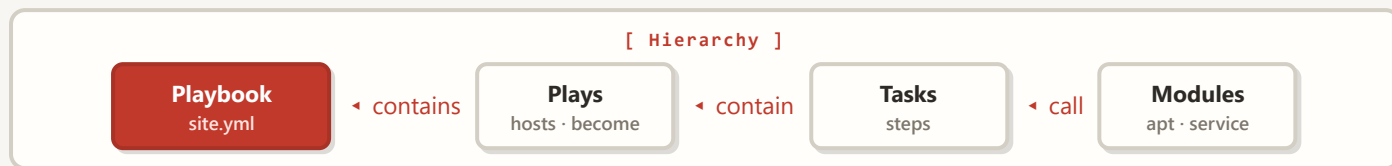
[**WATCH OUT**] In the Dictionary: notice that `name` and `age` are indented two spaces under `employee` — this Indentation is what makes them **inside it**. Align them at the same level and they become completely separate Key-Values.

🎓 **EXAM** The three golden YAML rules: ① **Start with** `---` · ② **Spaces, never Tabs** · ③ **Only three structures: Key-Value, Dictionaries, Lists**. Every later complexity in Playbooks is a composition of these three.

▶▶ The language is in your hands — the moment has come: your first real full Playbook → [Chapter 11](#)

Playbooks – Write & Run

A **Playbook** is a YAML file containing a set of **Plays**, and each Play contains a set of **Tasks** — to execute a complete automation scenario against a specific group of servers. This is Ansible's beating heart.



► TASK [Example: Playbook to install & run Apache] create site.yml

```

---
- name: Install and Start Apache Web Server
  hosts: web
  become: yes

  tasks:
    - name: Install Apache Package
      apt:
        name: apache2
        state: present

    - name: Ensure Apache Service is Running and Enabled
      service:
        name: apache2
        state: started
        enabled: yes
  
```

► TASK [Running the Playbook] execute it

```

$ ansible-playbook -i hosts.ini site.yml
PLAY [Install and Start Apache Web Server]
TASK [Gathering Facts] ok: [192.168.1.101]
TASK [Install Apache Package] changed: [192.168.1.101]
TASK [Ensure Apache Service is Running and Enabled] changed: [192.168.1.101]
PLAY RECAP
192.168.1.101 : ok=3 changed=2 failed=0
  
```

[**DEBUG · Line by line**] `hosts: web` → target the web group from the Inventory · `become: yes` → run with root privileges · every Task = a descriptive **Name** + a **Module** + the desired **State**. Notice the phrasing: we don't say "install" — we say "make it present."

[**PROOF**] **Run it a second time now.** The result: `changed=0` with every Task `ok` — Idempotency alive before your eyes: the state is already satisfied, so nothing changes. ♻️

► We installed and ran — clean removal and professional result reading: → [Chapter 11:b](#)

Playbooks – Removal & Results

The same power that installs also removes — just by flipping the State. Then we master reading the three result indicators.

► TASK [Playbook to stop & remove Apache] reverse order

```
---
- name: Stop and Remove Apache Web Server
  hosts: web
  become: yes

  tasks:
    - name: Stop Apache Service
      service:
        name: apache2
        state: stopped

    - name: Remove Apache Package
      apt:
        name: apache2
        state: absent
```

[KEY IDEA] The reverse logical order: **we stop the Service first**, then remove the Package — deleting a package while its service is running = unpredictable behavior. And the whole secret is in `state`: `present` / `absent` / `started` / `stopped` — four values that rule the world.

► TASK [Result indicators] the three colors

Indicator	Color	Meaning
ok	Green	The server is already in the Desired State — no change was required.
changed	Yellow	The Change was applied successfully to the server.
failed	Red	An Error occurred in the Task — execution stopped on that server.

[RESCUED • If you see failed] Read the red Task name first — it tells you **which step** failed, then the line under it — it tells you **why** (network? permissions? wrong package name?). 90% of failures are solved from these lines without any external search.

🎓 **EXAM** The essential difference between ok and changed is **the core of Ansible's philosophy**: ok = the state was already satisfied (Idempotency working), changed = real work was just done. And reading failed starts from the Task name + the Error Message below it.

► One file completed a full scenario — but as projects grow, that file becomes a monster. The final organization: → **Chapter 12: Roles & Galaxy**

Roles & Galaxy – Organization & Sharing

As automation projects grow, keeping all tasks and instructions in one Playbook file becomes **hard to maintain**. **Ansible Roles** give you a standard folder structure to organize the project and split it into **Modular & Reusable** components.

► TASK [The Role's folder structure] role anatomy

```
my_role/
├─ tasks/      # main.yml – the main tasks
├─ handlers/   # conditional tasks (Handlers)
├─ templates/  # Jinja2 templates
├─ files/      # static files to be transferred
├─ vars/       # variables specific to the Role
├─ defaults/   # default variables (overridable)
└─ meta/       # info about the Role & dependencies
```

[KEY IDEA] A Role = **Encapsulation**. Instead of a 200-line Playbook, you call a Role by name — and everyone reading the project understands the structure instantly, because everyone knows this standard layout.

► TASK [The Ansible Galaxy platform] community hub

An official open-source repository that lets you **share and download** ready-made **Roles** and **Collections** built by the Ansible community — to speed up development and automation. Instead of building an Apache Role from scratch: search Galaxy, download, and adjust to your needs.

galaxy.ansible.com
[Ready-made Roles](#)
[Collections](#)
[Community-driven](#)

[WATCH OUT] When to write a Role instead of a Playbook? Practical rule: your first Playbook = a regular Playbook. When the file starts repeating, or you find the same Tasks across multiple projects = time to split into Roles. Don't start with complexity.

🔥 **EXAM** Gather the book's whole message into one sentence — the closing interview question: "What is Ansible?" → an **Agentless Automation** tool over SSH, with **YAML Declarative Syntax** for the Desired State, characterized by **Idempotency**, organized with Inventory and Playbooks, scaling with Roles and Galaxy.

🔗 In closing

This guide summarizes all the fundamentals and practical concepts taught in the course by the **Cloud Code** channel — and can be relied on as a quick written reference while writing and executing automation scenarios. Then apply it: one Virtual Machine as a Control Node + one managed server, and rebuild every example with your own hands. Automation is mastered by practice, not reading. ►



Test Yourself — 12 Questions

Cover the answers until you've solved all twelve. 10+ correct = practically ready.

1. What are the two primary domains Ansible is used for?

2. What's the only requirement on Linux Managed Nodes?

3. On which Node is Ansible installed?

4. Define Idempotency in one sentence with an example.

5. What's the difference between Declarative and Imperative in Ansible?

6. Write the command to generate an SSH Key Pair of length 4096.

7. Which Key gets pushed to the Managed Node — and what happens if you push the other one?

8. Name the functions of the five core Modules: apt/yum, ping, service, copy, command/shell.

9. What do Inventory groups like [web] represent?

10. What's the default Module in Ad-hoc when -m is omitted?

11. What does changed=0 in the PLAY RECAP mean after the second run?

 **Honor question:** Ansible's four features + three domains + the full definition sentence — from memory.

[**DEBUG**] Questions cover chapters 1–12 in sequence. Any struggle → return to that chapter from the **TOC** before opening the answers.



Answer Key + Scoring

#	Answer	Why
1	Configuration Management + Infrastructure Provisioning	The two primary domains — the third: Network Automation
2	Python only	Pre-installed on most distros — no Agents
3	The Control Node only	Managed Nodes receive over SSH without installation
4	Running the same script repeatedly changes nothing unless a change is actually needed — example: Apache already installed → no change	Desired State satisfied → ok
5	Declarative in Playbooks (you define the State) · Imperative in Ad-hoc Commands (instant command)	Two approaches for two situations
6	<code>ssh-keygen -t rsa -b 4096</code>	-t the type (rsa) · -b the length (4096 bit)
7	id_rsa.pub — the Public Key · pushed the Private? Connection fails + a security hole	Public gets pushed; Private stays secret
8	apt/yum = Packages · ping = Connectivity · service = Services (start/stop/restart) · copy = Files · command/shell = Direct commands	The core five — everything else builds on them
9	The IPs of the Managed Nodes — Groups = targeting units referenced by name	We target [web] or all instead of IPs
10	command	The default when -m is absent
11	The state was already satisfied — Idempotency worked 	Same Playbook twice = zero Changes

[**PROOF** · 🔥 **Honor question**] The four features: **Agentless** · **Simple YAML Syntax** · **Declarative Approach** · **Idempotency** — the three domains: **Configuration Management** · **Infrastructure Provisioning** · **Network Automation**. And the full sentence: an Agentless Automation tool over SSH, with YAML Declarative syntax for the Desired State, Idempotent, organized with Inventory and Playbooks, scaling with Roles and Galaxy. Answered from memory? You understand Ansible — you don't memorize it.

► TASK [Score yourself] scoring

10–12 · 🏆 **Practically ready**

Move on to building real Roles and a full automation project.

7–9 · 🍌 **Solid**

Re-read the chapters you missed, then re-test.

<7 · 📺 **Replay**

Run the book's examples by hand on a real lab — doing sticks better than reading.

The Cheat Sheet — print & pin it

- ▶ **FULL FLOW:** keygen → ssh-copy-id → inventory → ad-hoc/playbook → recap
- ▶ **ARCHITECTURE:** Control Node —SSH→ Managed Nodes (Agentless · Python only)
- ▶ **STRUCTURE:** Playbook → Plays → Tasks → Modules · scale up: Roles → Galaxy

[⚙️ Setup Commands]

```
$ brew install ansible
$ sudo apt update && sudo apt install ansible -y
$ pip3 install ansible
$ ansible --version
$ ssh-keygen -t rsa -b 4096
$ ssh-copy-id -i ~/.ssh/id_rsa.pub user@ip
$ ssh user@ip
```

[⚡ Ad-hoc Syntax]

```
ansible <group> -i hosts.ini -m <module> -a "args"
ansible all -i hosts.ini -m ping
ansible web -i hosts.ini -a "uptime"
ansible all -m apt -a "name=nginx state=present" --become
```

[📄 Playbook Structure]

```
---
- name: scenario name
  hosts: web
  become: yes
  tasks:
    - name: task name
      apt:
        name: apache2
        state: present

# run:
ansible-playbook -i hosts.ini site.yml
```

[🚩 State Values]

present / absent	Package: installed / removed
started / stopped	Service: running / stopped
enabled: yes	Starts at boot
ok satisfied · changed applied · failed errored	

```
my_role/ tasks/ · handlers/ · templates/ · files/ · vars/ · defaults/ · meta/ # + galaxy.ansible.com
```

Agentless

Simple YAML

Declarative

Idempotent

Inventory Groups

Roles + Galaxy

[KEY IDEA · Closing line] You write the Desired State once — Ansible enforces it across all your Managed Nodes, and touches nothing if no change is needed. That's automation as it should be. ►